

Thinker AI

Complete Architecture & Brand Specification

Full Edition — June 2026

This document is the single source of truth for Thinker AI. It covers the complete V1 system architecture (Sections 1–27) and the Brand Identity Guidelines (Section 28). All sections are self-contained and cross-referenced.

Table of Contents

1. Purpose of This Document
2. V1 Scope Statement
3. Core Philosophy → Implementation Mapping
4. System Architecture ★ REVISED
5. Agent Specifications ★ REVISED
6. Model & Cost Strategy ★ REVISED
7. Data Flow & State ★ REVISED
8. Agent Fallibility & Error Containment ★ REVISED
9. Pipeline Observability & Post-Delivery Feedback
10. Technology Stack
11. Build Order ★ REVISED
12. V1 Launch Checklist
13. Out of Scope for V1
14. Open Decisions for Founder Review
15. Goal Intelligence System
16. Output Pipeline — Preview Before Build
17. Multi-Language Support
18. Paid Plan Architecture
19. Resilience — Continuous Delivery Guarantee
20. Dynamic Routing Engine
21. Think Credits, Pricing & Thinking Levels
22. File Upload Architecture
23. UI/UX Specification

- 24. API & Webhook Specification
- 25. Testing & QA Strategy
- 26. Error Codes & Messages
- 27. Analytics & Business Metrics
- 28. Brand Identity Guidelines

1.

Purpose of This Document

The three founding constitutions describe Thinker AI's vision and philosophy: a multi-agent system that understands goals, plans work, debates solutions, and delivers verified outcomes. They do not specify how to build it. This document translates that philosophy into a concrete, buildable V1 architecture defining every agent, the data that flows between them, the technology choices, and the order of implementation.

Guiding constraint: all twelve agent roles are implemented with the minimum logic needed to fulfil each role. Complexity is added in later versions, not V1.

2.

V1 Scope Statement

2.1 What V1 Is

A web-based chat application where a user describes a goal and a dynamic pipeline of thirteen lightweight agents — coordinated by Thinker Core — clarifies the request, thinks strategically, plans it, builds a deliverable, reviews it, and returns one final answer. Thinker AI supports 100+ languages automatically.

2.2 What V1 Is Not

- Not a fully autonomous multi-day project system.
- Not a mobile app (V1 ships as a responsive web app).
- Not running every request through full multi-model debate — consensus is invoked selectively.
- Not retaining credentials, passwords, or payment data in memory.

2.3 V1 Success Criteria

#	Criterion
1	User can chat normally and get direct answers without triggering the full pipeline
2	User can request a project and the system asks clarifying questions before building
3	All thirteen agent roles are present in code
4	Every final deliverable passes through Reviewer before being shown to the user
5	Multi-model consensus works end-to-end for Judge Agent
6	Project memory persists within a session and reloads on return
7	Visual assets require explicit user approval before project is marked complete
8	Smart Clarification detects vague goals and activates 3-Level Deep mode
9	Signature Question fires on every project-type request before Builder runs

3.

Core Philosophy → Implementation Mapping

Constitution Principle	V1 Implementation
Understand first, plan second, execute third, verify always	Dynamic pipeline order enforced in code — but routing adapts on failure
Requirement Clarity Law — ask minimum questions before any work	Significant work hard gate — Builder cannot run until requirements are complete
Token Economy / Resource Protection Law	Cheap single-model calls by default; multi-model consensus only for high-stakes decisions
One Goal. One Team. One Final Answer.	Thinker Core merges all agent outputs into exactly one response object
No single model/agent/opinion is final — verified code is final	Code requires approval required before Thinker Core marks a deliverable final

The Founder Law

Thinker AI must not immediately build what the user asks for. It must first determine whether the user's goal, problem, and assumptions are valid. No code is ever written before the thinking is done.

The Preview Before Build Law

Thinker AI must show the user what it intends to build before generating the final deliverable. User approval is required at each stage before the next stage begins.

4.

System Architecture ★ REVISED

4.1 Pipeline Overview — Dynamic Routing

The pipeline no longer runs in a fixed 1→13 sequence. Thinker Core operates as a dynamic routing engine. After every agent completes, Thinker Core reads the agent's `next_action` output field and decides the next step.

Condition	next_action	Thinker Core Response
Simple chat / general question	direct_answer	Skip pipeline → single LLM call → return
Request is vague or incomplete	clarify	Halt → return questions to user
Requirements complete, project requested	proceed	Run full pipeline in order
Builder output rejected by Reviewer	retry	Re-run Builder (same plan, same step)
Reviewer + Critic both flag deep issue	replan	Return to Planner with failure context
Planner→Builder loop exceeds 2 cycles	clarify	Return to Clarification for new requirements
Judge score borderline	escalate	Invoke Consensus Agent
All models for an agent unavailable	failover	Model Pool Manager assigns next provider
Any agent exceeds loop_count limit	halt	Pause pipeline, notify user, save state

4.2 Thinker Core Responsibilities ★ REVISED

- Owns the pipeline state machine and dynamic routing table for each conversation.
- Dynamic next-step decision: reads `next_action` from every agent output before passing control.
- Model Pool Manager: maintains a pool of available models per agent tier. If a model fails or times out (5s threshold), silently reassigns to the next available model.
- Loop detection: tracks `loop_counts` per agent. If any agent is retried more than 3 times, or Planner→Builder→Reviewer cycles more than 2 times, halts and notifies user.
- Peer audit trigger: when a failover model takes over, instructs it to audit what the previous model produced before continuing.
- State preservation on halt: writes full Pipeline State to database before stopping. Resume always possible from the last completed agent step.
- Enforces the Clarification gate and merges all agent outputs into one final response.

4.3 Builder vs. Design Agent Routing

Planner Step Output Type	Routed To
code, text, config	Builder Agent
image	Design Agent

4.4 Domain Picker

Named domain agents (Music, DevOps, Security, Coding, etc.) provide a domain hint to the pipeline — they are not alternate paths. Intent Agent is skipped when a domain is selected. Planner and Builder receive the domain as prompt context.

Implementation gap: the domain picker exists in the UI but the selected domain is not yet passed to `/api/pipeline`. Closing this gap is Phase 10 in the build order.

5.

Agent Specifications ★ REVISED

All agents now include one additional output field read exclusively by Thinker Core: next_action (proceed | retry | replan | clarify | escalate) and reason (string). Agents recommend next_action but do not execute it.

5.1 Intent Agent

Role	Classify the incoming message. Also outputs domain label and Thinking Level. Skipped when user selects a goal.
Input	Raw user message + last 5 messages of context.
Output	Intent type, confidence score, domain, thinking_level.
V1 Implementation	Single LLM call with classification prompt.
Model Tier	Fast/cheap.

5.2 Clarification Agent ★ UPDATED

Role	Enforce Requirement Clarity Law AND run Smart Clarification — detect emotional context, surface hidden goals.
Input	Intent type, full conversation, emotional tone signals.
Output	Completeness flag, smart questions, known requirements, assumption flags, Signature Question response.
V1 Implementation	LLM call against Smart Clarification Layer + 3-Level Deep + Constraint & Assumption Discovery + Signature Question.
Model Tier	Fast/cheap with enhanced prompt engineering.

5.3 Planner Agent

Role	Convert a clarified goal into an ordered list of concrete steps using the Strategy Agent brief as context.
Input	Known requirements + Strategy brief + Signature Question response + Constraint findings.
Output	List of steps: id, description, output_type, dependencies.
V1 Implementation	Single LLM call. No re-planning loop in V1.
Model Tier	Mid-tier reasoning.

5.4 Research Agent

Role	Gather external information when a step requires it. Optional — only invoked if Planner flags a step.
Input	Single Planner step flagged as needing research.
Output	Findings and sources.
V1 Implementation	Uses web search. Skipped for purely generative steps.
Model Tier	Fast/cheap with search.

5.5 Builder Agent

Role	Produce the actual deliverable — code, copy, configuration, or content — for each Planner step. Most token-heavy.
Input	Step description, research findings, relevant project memory.
Output	Artifact type, content, generated files.
V1 Implementation	Runs once per step, sequentially in V1.
Model Tier	Strongest available.

5.6 Reviewer Agent

Role	Run QC checklist: accuracy, logic, security, completeness, user-goal alignment. For images: relevance and appropriateness.
Input	Builder output + original requirements.
Output	Pass/fail flag and list of issues with severity.
V1 Implementation	Single LLM pass. If fails with high-severity issues, returns to Builder once — capped at one retry.
Model Tier	Mid-tier reasoning.

5.7 Critic Agent

Role	Independently look for weaknesses Reviewer may miss. Critic's prompt withholds Reviewer's verdict until after Critic's findings.
Input	Builder output + Reviewer findings.
Output	List of concerns with overall severity rating.
V1 Implementation	Single LLM call with adversarial/skeptical framing — different persona from Reviewer.
Model Tier	Mid-tier reasoning.

5.8 Judge Agent

Role	Score on accuracy, feasibility, safety, cost, completeness. For high-risk categories (auth, payment, data deletion), also includes legal compliance.
Input	Builder output, Reviewer result, Critic concerns.
Output	Score per criterion and approval decision.
V1 Implementation	If clearly high or clearly low, Judge decides alone. If borderline, invokes Consensus Agent.
Model Tier	Strongest available.

5.9 Consensus Agent

Role	Multi-model consensus used only when Judge needs a tie-breaker or for mandatory high-risk categories. Three models vote.
Input	Same question Judge was uncertain about.
Output	Each model's vote and reasoning, plus final verdict.
V1 Implementation	Sends identical question to 3 different providers in parallel. Majority vote wins; ties default to conservative outcome.
Model Tier	Three different providers.

5.10 Design Agent

Role	Handle visual/graphic output — logos, illustrations, UI imagery, icons. Does not attempt 3D models or animations.
Input	Planner step flagged as output_type: image + style/brand requirements.
Output	Generated image files + short text description.
V1 Implementation	Single call to external image-generation API. Output passes through Reviewer.
Model Tier	External image-generation provider.

5.11 Visual Asset Approval Flow

Role	Images follow a different path: (1) Clarification asks one style question. (2) Generated image shown to user with approval gate.
Input	N/A — this is a flow, not a single agent.
Output	User approval or revision instruction.
V1 Implementation	Approval gate on every generated image before project marked complete.
Model Tier	N/A.

5.12 Strategy Agent

Role	Business, product, and market thinking before technical planning. Runs after Clarification, before Planner, on
Input	Clarified requirements + domain hint + Signature Question response + Constraint findings.
Output	Strategy brief: goal clarity, MVP scope, key risks, success criteria.
V1 Implementation	Single LLM call with structured prompt for business/product thinking.
Model Tier	Mid-tier reasoning.

5.13 Document Agent ★ NEW

Role	Read, parse, and extract structured content from any document file. 13th agent role.
Input	Uploaded file (PDF, DOCX, TXT, MD, CSV, XLSX) + user intent + conversation context.
Output	Structured content: {summary, key_points, sections[], data_tables[], entities[], language, page_count}.
V1 Implementation	Two-step: (1) file parsing library extracts raw text; (2) single LLM call structures the output.
Model Tier	Fast/cheap.

5.2.1 Goal Discovery Mode

Layer	Purpose	Example Question
1. Goal Discovery	What does the user actually want to achieve?	"Why do you want to build this?"
2. Problem Discovery	What specific problem is being solved?	"What problem will this solve?"
3. Audience Discovery	Who is this for?	"Who will use this?"
4. MVP Discovery	Minimum viable starting point	"Which 3 features if you could only launch with 3?"
5. Success Discovery	What does success look like?	"How will you know this worked in 90 days?"

5.2.2 Smart Clarification Layer

- Emotional Context Detection: reads tone, not just words. Softens language when stress or urgency is detected.
- Contradiction Detection: catches conflicts between later requests and earlier stated constraints.
- Assumption Surfacing: identifies unstated assumptions and confirms them before Planner locks scope.
- Progressive Disclosure: questions adapt based on previous answers — irrelevant follow-ups are skipped.
- Intent Confidence Threshold: 90%+ → proceed; 70–90% → one confirmation; below 70% → Goal Discovery Mode.

5.2.5 Signature Question Protocol

"Why do you believe this is the right solution?" — asked on every project-type request without exception, before Builder is ever called. Hard rule: Clarification Agent never blocks a user who insists on proceeding — the user's autonomy is always respected.

Model & Cost Strategy ★ REVISED

6.1 Model Tiers ★ REVISED

Tier	Used By	Purpose	Pool Size	Provider Rule
Fast/cheap	Intent, Clarification, Research, Debugging	High frequency, low-complexity	8 models	Min 2 providers
Mid-tier	Planner, Strategy, Reviewer, Critic	Structured reasoning	3 models	Min 2 providers
Strongest	Builder, Judge	Highest-stakes generation	3 models	Min 3 providers
Image generation	Design Agent	Visual asset generation	2 models	Min 2 providers
Multi-model	Consensus Agent	Tie-breaking votes	3 models	Exactly 3 providers

6.2 Resource Protection Law in Practice

- Every pipeline run is logged with token cost per agent.
- Clarification gate prevents Builder (most expensive) from running on unclear requests.
- Consensus Agent invoked only when Judge's confidence is borderline — under 20% of all requests.
- Reviewer retry loop capped at one retry to prevent runaway cost.
- Think Credits make cost visible and user-controlled — no silent overruns.

6.4 When Consensus Actually Fires

Judge Score	Action
Clearly passes all five criteria	Approve directly — no Consensus call
Clearly fails (safety or accuracy critically low)	Reject — send back to Builder
Borderline on any criterion	Invoke Consensus Agent for that criterion only
High-risk category (auth, payment, data deletion, perms)	Consensus fires automatically regardless of score

6.5 Model Identity Concealment

No provider or model name (Claude, GPT, Gemini, or any other) appears anywhere the user can see — not in chat, not in error messages, not in debug views. Each model slot is referred to by a generic label (Agent A, Agent B). Failure messages describe failures generically: 'one of the verification steps didn't respond' — never naming the provider.

Data Flow & State ★ REVISED

7.1 Pipeline State Object ★ REVISED

All original fields retained. New fields added for dynamic routing and resilience:

New Field	Type	Purpose
current_agent	string	Which agent is currently executing
routing_history	array	{agent, next_action, reason, model_used, timestamp}
loop_counts	object	{planner: int, builder: int, reviewer: int, ...}
failover_log	array	{agent, failed_model, replacement_model, timestamp, peer_audit_result}
peer_audit_flag	boolean	Set true when failover model must audit previous output
resume_from_agent	string null	Set on halt — identifies restart point on resume
signature_question_response	string null	User's answer to Signature Question or null if declined
constraint_findings	object	Time, budget, skill, network, existing assets
assumption_flags	array	Surfaced assumptions and whether each was confirmed

7.2 Memory System

Memory Type	Scope	Storage	Retention
Short-term	Single conversation turn	In-memory / session	Cleared on session end
Project memory	One project across its lifecycle	Database, keyed by project	Until user deletes project
Long-term preference	User-level, across projects	Database, keyed by user	Indefinitely unless cleared
Decision Memory	Explicit saved decisions	Database, keyed by user	Until user revokes

Hard rule: passwords, PINs, API keys, and credentials are never written to any memory layer, even temporarily.

7.3 Decision Memory

Set explicitly — user states a deliberate rule like 'always use React' and the system follows it automatically. When a user states a rule (Clarification Agent detects phrasing like 'always', 'never', 'from now on'), Thinker Core writes it to Decision Memory and confirms. On every subsequent project, relevant decisions are injected into Planner context. If a new request conflicts with a saved decision, Thinker Core surfaces the conflict explicitly rather than silently overriding.

Agent Fallibility & Error Containment ★ REVISED

8.2 Containment Rules

Rule	What It Prevents
No single agent's approval is final — Judge approval always required on top of Reviewer	As single agent verdict reaching the user unchecked
If Reviewer and Critic disagree, Thinker Core routes to Consensus rather than letting Judge break silently	Arbitrary Judge getting away with anything resolved by whichever agent runs last
Critic's prompt withholds Reviewer's verdict until Critic has formed its own Opinion	Opinion anchoring on 'Reviewer said it's fine' and skipping its own check
High-risk categories (auth, payment, data deletion, permissions): Consensus required across same-family models approving dangerous code	Shared blindspot across same-family models approving dangerous code
Every rejection or override is logged with both verdicts and reasoning	Loss of visibility into how often agents disagree

8.4 Peer Recovery Protocol ★ NEW

- Step 1 — Failure Detection:** Thinker Core detects failure when a model returns an error, times out after 5 seconds, or returns malformed output.
- Step 2 — Context Handoff:** Thinker Core passes the full Pipeline State to the replacement model, including partial output and peer_audit_flag set to true.
- Step 3 — Peer Audit:** Replacement model reviews what failed model produced — identifies what is correct, incorrect, and missing. Audit result written to failover_log.
- Step 4 — Targeted Continuation:** Replacement generates only what is needed — keeps correct partial output, fixes incorrect, completes missing. Never regenerates correct work.
- Step 5 — Normal Pipeline Resumes:** Replacement model's output treated identically to a first-attempt output.

8.5 Loop Detection Rules ★ NEW

Loop Type	Threshold	Response
Single agent retry	3 attempts	Halt. Save state. Notify user.
Planner → Builder → Reviewer cycle	2 full cycles	Return to Clarification with failure context.
Clarification loop (vague answers)	4 rounds	Surface the specific unknown directly.
Judge → Consensus → reject cycle	2 cycles	Halt and present best available output with quality caveats.

Pipeline Observability & Post-Delivery Feedback

9.2 Per-Agent Logging

Field	What It Records
agent_name	Which agent ran
input_summary	Short description of what this agent received
output_summary	Short description of what this agent produced
duration_ms	How long the agent took in milliseconds
status	success failed skipped
error_detail	If failed — error message using generic terms, never provider names
confidence	Judge's score, Consensus vote breakdown, Reviewer pass/fail
retry_count	How many times this agent was retried
failover_cost	Token spend from failover calls, separate from normal pipeline cost
clarification_depth	Which Clarification layers ran (Clarification Agent only)
signature_q_answered	Whether user answered the Signature Question

9.4 Post-Delivery Feedback Loop

User's Problem	System Response
Bug in the output	Ask user to describe bug. Send back to Builder with bug + original plan. Reviewer re-runs after fix.
Missing feature	Treat as new small requirement. Run Clarification → Planner (missing piece only) → Builder → Reviewer.
Style not to taste	For images: Visual Asset revision flow. For code: ask for specific style feedback, send to Builder.
Vague ('it's bad')	Ask one question: 'Is there a specific feature broken, something missing, or does it look wrong?'

9.5 Fix Limits — Cost-Based

Fix Type	What Runs	Limit
Small (change a color, update a label)	Builder only	Unlimited
Medium (fix a bug, add a feature)	Builder + Reviewer + Critic	Up to 10 per project session
Full rebuild (start over from requirements)	Full pipeline from Planner	Up to 3 per project session
Rollback (restore a previous version)	Database lookup — no agent runs	No limit — always free

9.6 Version History

Every time Builder produces an output, Thinker Core saves that version before overwriting the current state. Only the last 10 versions are kept per project. Rollback restores a previous version instantly with no rebuild and never counts against any fix limit.

10.

Technology Stack (V1 Recommendation)

Layer	Choice	Reasoning
Frontend	React + Vite	Matches existing developer experience
Backend	Node.js (Express) or Firebase Cloud Functions	Lightweight, matches existing Firebase familiarity
Database	Firestore	Document model fits the pipeline state object well
Auth	Firebase Auth	Reuse existing pattern
LLM Access	Direct API calls to each provider, backend only	Protects API keys, matches Resource Protection Law
Payments	Stripe	Webhook-based plan tier updates; never touches raw card data
Hosting	Firebase Hosting or Vercel	Simple, matches existing deployment habits

11.

Build Order ★ REVISED

Phase	Deliverable
1	Thinker Core skeleton + direct chat path (passthrough)
1.5 ★	Dynamic Routing Engine: routing table, next_action handling, loop_counts, loop detection, Model Pool Manager skeleton
1.6 ★	Model Pool Manager — failover detection (5s timeout), pool health monitoring, Peer Recovery Protocol, failover_log
2	Intent Agent + Clarification Agent (basic) + hard gate
2.5	Smart Clarification Layer (Section 5.2.2)
2.6	3-Level Deep Clarification + Signature Question (Sections 5.2.3, 5.2.5)
2.7	Constraint Discovery + Assumption Discovery (Section 5.2.4)
3	Planner Agent + Builder Agent (single-step)
4	Reviewer Agent + one-retry loop back to Builder
5	Critic Agent + Judge Agent (Judge decides alone)
6	Consensus Agent + borderline-routing from Judge
7	Research Agent (optional step augmentation)
8	Design Agent + Builder/Design routing
8.5	Strategy Agent
9	Project memory + long-term preference + Decision Memory
10	Domain picker wiring

Phase	Deliverable
10.5	Per-agent logging + post-delivery feedback loop
10.6 ★	Resilience observability: failover_log dashboard, pool health alerts, routing_history trace viewer
10.7 ★	Think Credits integration: credit deduction engine, balance checks, confirmation prompts
10.8 ★	Document Agent + File Upload pipeline (Section 22)
11	Polish: UI, error states, retry UX, and the launch checklist

V1 Launch Checklist

- Stable chat experience — direct path works without errors
- Full agent pipeline completes end-to-end on app, website, and game project types
- Strategy Agent produces a meaningful brief visibly reflected in Planner's output
- Decision Memory saves, applies automatically, and surfaces conflicts correctly
- Design Agent produces a usable image; approve/reject/revise all work; 3-attempt cap enforced
- Clarification gate cannot be bypassed
- Smart Clarification detects emotional tone and adjusts question framing
- Contradiction Detection catches conflicts between later messages and earlier constraints
- Signature Question fires on every project-type request; response stored in Pipeline State
- Intent Confidence Threshold correctly routes: high → proceed, mid → confirm, low → Goal Discovery
- No credentials ever appear in stored memory (manual audit)
- Consensus Agent tested with intentionally borderline cases
- High-risk categories (auth, payment, data deletion, permissions) trigger Consensus automatically
- Token cost per request is logged and reviewed
- Final response is always a single merged answer — never raw agent dumps
- 5-stage output pipeline works end-to-end (Section 16)
- Language detection works — user writes in any Tier 1 language, all responses in that language
- RTL language (Arabic or Hebrew) renders correctly
- Per-agent log entries written for every pipeline run; failed run traceable to specific agent
- Cost-based fix limits enforced: small unlimited, medium capped at 10, full rebuild capped at 3
- Version history saves correctly; rollback restores without Builder call; 10-version limit enforced
- Dynamic routing engine tested — all `next_action` values handled correctly
- Failover tested — model timeout triggers pool reassignment within 5 seconds
- Peer Recovery Protocol tested — replacement model audits and continues partial work correctly
- Loop detection tested — exceeding thresholds triggers halt and state save
- Think Credits deducted correctly; no silent overruns; confirmation shown above 3 credits
- Stripe webhook correctly updates plan tier on payment and downgrade
- Free Trial 50-credit one-time limit enforced; no monthly refill
- File upload security rules tested — executables, oversized files, credentials all blocked
- Document Agent correctly structures output from PDF, DOCX, and TXT uploads
- No provider/model names reachable from any user-facing surface including error states

13.

Explicitly Out of Scope for V1

- Parallel multi-step Builder execution
- Automatic re-planning when a step fails repeatedly
- Native mobile app (Play Store launch can wrap the web app initially)
- Organisational/team-shared memory
- Autonomous multi-day project system with no human checkpoint
- Native Thinker foundation model (V1 uses third-party model APIs only)
- 3D model generation
- Animation and video generation
- Interactive prototypes or Figma-format design file output
- Live API testing in preview (V2 feature)

14.

Open Decisions for Founder Review

1. Consensus uses exactly three providers — confirmed for V1, configurable in later versions.
2. Reviewer retry capped at one loop — confirm this is acceptable or should be configurable.
3. Research Agent is optional and skippable per step — confirm this matches intended behaviour.
4. Technology stack assumes continuity with existing Firebase project — confirm versus fresh stack.
5. High-risk category list (auth, payments, data deletion, permissions) — confirm completeness.
6. Design Agent image provider not yet chosen (DALL-E vs Stability AI) — confirm based on cost and licensing.
7. Free Trial credit amount set at 50 (one-time) — confirm conversion funnel balance.
8. Pro price at \$19/month, Founder at \$59/month — confirm based on competitive positioning.
9. Critic Agent skipped on Free plan to reduce cost — confirm acceptable quality trade-off.

Goal Intelligence System

15.1 Goal Discovery

Goal Discovery works through five layers of questions — one at a time, conversationally — to surface the real goal beneath the stated one. Goal Discovery is not an obstacle — it is the product. A user who spends 5 minutes in Goal Discovery is far more likely to end up with something useful.

15.3 Idea Validation

Strategy Agent runs an Idea Validation covering four questions: (1) Is the problem real? (2) Is this idea the right solution? (3) Is this the right time? (4) Why this team? The output is a brief that sharpens the plan — not a go/no-go block.

15.5 Founder Mode

Normal Mode	Founder Mode
Goal clarity for this build	Problem → Market → Competition → Business model
MVP scope (features)	MVP scope (minimum to validate the business)
Key technical risks	Full risk landscape: market, technical, financial, execution
Success criteria for deliverable	Success criteria for the business itself

Go / Caution / No-Go assessment is always advisory, never a block. Activated explicitly ('Activate Founder Mode') or automatically when request involves market, revenue, competition, or business model language.

16.

Output Pipeline — Preview Before Build

Stage	Name	User Sees	User Action
1	Thinking Summary	Goal understanding, strategy brief, proposed approach	Confirm or correct
2	Blueprint Preview	Full plan: screens/components, tech stack, complexity	Approve, edit, or request changes
3	Build Preview	Real-time progress + live rendered UI. Mandatory approval for first screen	Approve first screen, or change direction
4	Approval	Full build visible in preview panel	Approve, change something, or go back
5	Final Output	Project summary, Live Preview link, ZIP download, ArchDoc download	Download and use

Simple requests skip stages 2 and 4. The user is never surprised by what was built — even for simple outputs, Thinker Core shows a one-line summary and waits for brief confirmation before running Builder.

17.

Multi-Language Support

17.2 What Gets Translated vs. What Stays in English

Content Type	Language	Reason
All conversation with user	User's language	Product interface
Clarification questions, strategy brief	User's language	Must be clearly understood
Progress indicators, error messages	User's language	UX must be native-feeling
Generated code (variables, functions)	English	Industry standard
Code comments	User's language or English (user's choice)	Developer's Decision Memory
README and documentation	User's language	User needs to understand their own project
Folder/file names, API endpoint names	English	Compatibility and portability

17.4 Supported Language Tiers

Tier	Languages	Quality
Tier 1 — Full	English, French, Spanish, German, Portuguese, Italian, Dutch, Japanese, Korean, Chinese, Arabic, Hindi, Bengali, Turkish, and others	High Quality; Seamless, Accurate
Tier 2 — Good	50+ additional languages including Urdu, Malay, Thai, Swahili, Romanian, Czech, Polish, Vietnamese, and others	Good Quality; Occasional Errors, High Success
Tier 3 — Basic	Remaining languages supported by underlying model providers	Varies; user informed if quality may be limited

18.

Paid Plan Architecture

All plan enforcement happens inside Thinker Core before any agent is invoked. No billing or quota logic exists inside individual agents.

18.1 Plan Tiers

Tier	Price	Think Credits	Target User
Free Trial	\$0	50 (one-time, never refills)	Individuals exploring the product
Pro	\$19/mo or \$199/yr	1,500/month	Freelancers, indie builders
Founder	\$59/mo or \$589/yr	5,000/month	Power users, startup founders
Enterprise	Custom	Custom	Organisations 10+ seats (V2)

18.2 Feature Gate Matrix

Feature	Free Trial	Pro	Founder
Direct Chat	✓	✓	✓
Basic Clarification	✓	✓	✓
Smart Clarification (3-Level, Sig Q)	—	✓	✓
Strategy Agent	—	✓	✓
Founder Mode	—	✓	✓
Planner Agent	✓	✓	✓
Research Agent	—	Limited	Unlimited
Builder Agent	Credit-limited	Credit-limited	Credit-limited
Reviewer + Judge	✓	✓	✓
Critic Agent	—	✓	✓
Consensus Agent	—	—	✓
Design Agent	—	✓	✓
Document Agent	✓	✓	✓
5-Stage Output Pipeline	Stage 1+5 only	Full	Full
Version History	3 versions	10 versions	25 versions
Decision Memory	—	—	✓
Thinking Level: High (■)	—	✓	✓

Feature	Free Trial	Pro	Founder
Thinking Level: Consensus (■)	—	—	✓
Priority Queue	—	—	✓
API Access	—	—	✓
Extra credit rate	\$0.10/credit	\$0.07/credit	\$0.05/credit

18.3 Billing Integration

- Stripe Checkout handles all payment collection. Thinker AI never touches raw card data.
- On successful payment, Stripe webhook updates plan_tier in Firestore and refreshes session token.
- Downgrade takes effect at end of current billing period. Pro-created projects preserved but gated features become read-only.
- Failed payment: 7-day grace period of continued access while Stripe retries. After 7 days, tier reverts to Free Trial.
- Refunds handled entirely in Stripe — plan_tier change is the only signal Thinker Core needs.

Resilience Architecture — Continuous Delivery Guarantee

Thinker AI never holds a user's work hostage to a single model, a single provider, or a temporary outage. When any model fails, another model takes the task, audits what was done, and continues. The user's work always moves forward.

19.1 Failure Detection Timeline

Time	Event	Thinker Core Action
0s	Model call initiated	Start 5-second timeout clock
0–5s	Model responds normally	Continue pipeline — no action needed
5s	No response received	Mark model as failed. Assign next model from pool.
5–7s	Replacement initialises	Peer audit begins (Section 8.4 Steps 2–3)
7s+	Replacement continues	Pipeline resumes — user sees no pause
All pool models fail	Pool exhausted	Loop detection triggers halt. State saved.

19.2 User Communication During Failures

Situation	Message to User
Failover resolves within 7s	No message. Pipeline continues normally.
Failover takes 7–30s	'Still working on this — taking a moment longer than usual.'
Partial completion, halt	'I've completed [X of Y steps]. Your work so far is saved. Would you like me to try a different approach?'
All models unavailable	'I wasn't able to complete this step right now. Your work is saved and you can resume at any time.'

Hard rule: no provider or model name appears in any user-facing failure message.

Dynamic Routing Engine — Implementation

20.1 Routing Decision Logic (pseudocode)

```
function routeAfterAgent(agent_output, pipeline_state):
    action = agent_output.next_action
    log_routing_decision(agent_output, pipeline_state)
    if action == 'proceed':
        return dispatch(next_in_sequence(pipeline_state.current_agent), pipeline_state)
    if action == 'retry':
        pipeline_state.loop_counts[current_agent] += 1
        if loop_counts[current_agent] > 3: return trigger_halt(pipeline_state)
        return dispatch(current_agent, pipeline_state)
    if action == 'replan':
        pipeline_state.loop_counts['planner_cycle'] += 1
        if loop_counts['planner_cycle'] > 2: return dispatch('clarification', pipeline_state)
        return dispatch('planner', pipeline_state_with_failure_context)
    if action == 'clarify': return dispatch('clarification', pipeline_state)
    if action == 'escalate': return dispatch('consensus', pipeline_state)
    if action == 'direct_answer': return dispatch('direct_llm_call', pipeline_state)
```

20.2 Model Pool Manager Logic (pseudocode)

```
function callAgentWithFailover(agent_name, input, pipeline_state):
    pool = get_pool(agent_name) # from pool_health_snapshot
    for model in pool.available_models:
        try:
            result = call_model(model, input, timeout=5s)
            if is_valid_output(result): return result
        except TimeoutError, ModelError:
            log_failover(agent_name, model, pipeline_state)
            next_model = pool.next_available(exclude=failed_models)
            if next_model:
                input_with_audit = add_peer_audit_flag(input, partial_result)
            else:
                return trigger_halt(pipeline_state) # pool exhausted
```

20.3 What the Routing Engine Is Not

- Not an LLM call — plain backend logic (conditional branching on next_action values).
- Not visible to agents — agents receive only their defined inputs.
- Not a replacement for good agent design — it handles failures, not poor prompts.
- Not a billing layer — plan tier checks (Section 18.3) run before the routing engine.

21.

Think Credits, Pricing & Thinking Levels

More Thinking
= More Questions
= More Validation
= More Credits

Users do not pay for answers. They pay for the depth of thinking behind the answer. Every interaction — including simple chat — costs at least 1 credit because every interaction invokes a third-party model API call.

21.1 Thinking Levels

Level	Icon	Purpose	Agents Active	Base Cost
Low	■	Fast answers, simple chat	Intent + Direct Answer	1 credit
Medium	■	Analysis, comparisons	Intent + Clarification + Planner + Builder	3 credits
High	■	Deep planning, full project builds	Full pipeline (all 13 agents)	10 credits
Consensus	■	Multi-model validation	Full pipeline + Consensus (3 providers)	30 credits

21.2 Plan Restrictions by Thinking Level

Thinking Level	Free Trial	Pro	Founder
■ Low	✓	✓	✓
■ Medium	✓	✓	✓
■ High	—	✓	✓
■ Consensus	—	—	✓

21.3 Credit Cost Table

Action	Credits	Notes
Simple chat / direct answer	1	Per message. Always applies.
Clarification reply (per round)	2	Each round of questions + user answer
Research Agent (per step)	3	Only when Planner flags a step as research-needed
Medium analysis	4	Medium Thinking sessions
Image generation (per image)	10	Per image, per attempt including revision
High Thinking session (base)	10	Charged once at session start
Consensus session (base)	30	Charged once at session start
Document analysis (per file)	3	When Document Agent completes

Action	Credits	Notes
Code review (per file)	5	When Reviewer + Critic complete
Code fix / improvement	10	When Builder completes the fix
ZIP extract + structure map	5	When Thinker Core maps the structure
Image edit / object removal	15	Per attempt
Image style transfer	15	Per attempt
Reference-based image generation	20	Per attempt
Small fix (Builder only)	1	Color change, label update
Medium fix (Builder + Reviewer + Critic)	5	Bug fix, missing feature
Full rebuild (Planner onwards)	50	Complete restart from planning
Rollback (any version)	0	No LLM call — database lookup only
Failover (model substitution)	0	Absorbed by operator — never charged to user

File Upload Architecture

Thinker AI accepts any file a user uploads. Thinker Core identifies the file type, routes it to the correct agent, and determines what to do — the user does not need to specify 'review this code' or 'analyse this PDF.' Thinker reads the file and asks the right question.

22.1 Supported File Types & Routing

Category	Formats	Max Size	Routed To	Credits
Documents	PDF, DOCX, TXT, MD	50 MB	Document Agent	3/file
Code	JS, TS, PY, HTML, CSS, JSON, YAML, All code Reviewers	10 MB	Reviewers → Critic → Judge → Builder	5 review / 10 fix
Images	PNG, JPG, JPEG, SVG, WEBP, GIF	20 MB	Design Agent	2 analyse / 15 edit / 20 ref-gen
Data	CSV, XLSX, JSON	25 MB	Research Agent	3/file
Archives	ZIP, TAR	100 MB	Thinker Core (extract & re-route)	5 + per-file costs

22.2 Image Operations

Operation	User Intent Signal	Action	Credits
Analyse	No edit instruction	Vision model describes content	2
Edit / remove	'Remove background' / 'Delete the person'	Image inpainting / masking API	15
Style transfer	'Make this cartoon style'	Style transfer API applied to full image	15
Reference-based	'Use this logo's style to make a banner'	Style extracted from reference; new asset generated	20
Upscale / enhance	'Make this higher resolution'	Upscaling API applied	8

22.3 File Security Rules

- Executable files (.exe, .bat, .sh, .dll, .bin, .dmg) are rejected before processing — no credits charged.
- No file stored permanently — uploaded files live in temporary in-memory workspace and are deleted when the session ends.
- File content sent to LLM providers as context only — never uploaded to separate file storage.
- Code files scanned for: shell injection, data exfiltration, cryptomining, known malware signatures.
- If a code file contains what appears to be an API key or password, Thinker Core warns the user and redacts the value from LLM context.

UI/UX Specification

23.1 Overall App Layout

Zone	Location	Contents
Left Sidebar	Fixed, collapsible	Logo, Domain Picker, Conversation History, New Chat, Upgrade CTA
Main Chat Area	Centre, scrollable	Message thread, Thinking Level indicator, progress stages, output panels
Input Bar	Bottom, fixed	Text input, file upload, Thinking Level selector, send button, credit balance chip
Right Panel	Slide-in on demand	Project details, version history, pipeline log (Founder), settings shortcuts
Top Bar	Fixed, minimal	Conversation title (editable), domain label, user avatar, notifications

23.2 Screen Layouts

Screen	Key Elements
Home	Greeting ('Good Evening') · 'What are we building today?' prompt · Bottom nav: Chat · Projects · Create · Files · Profile
Chat	Message thread · Thinking indicator between user msg and Thinker reply · Pipeline stage labels · Bottom: input + file upload
Agent View	Hidden by default, expandable · Shows live pipeline stages with checkmarks and credit cost per stage
Project	Project name + status badge · Timeline: Thinking → Planning → Building → Testing → Completed · Download ZIP · Version history
Voice	Full-screen dark · Logo centered with Core Pulse animation · Minimal wave animation · Tap anywhere to stop
Settings	8 tabs: General · Thinking & Pipeline · Memory · Files · Credits & Billing · Notifications · Privacy & Security · Advanced
Profile	Identity (name, email, photo) · Activity stats · Decision Memory manager · Linked accounts

23.3 Message Bubble Style

Sender	Background	Text Color	Alignment
User	#0B6E69 (Deep Teal)	#F9FAFB	Right-aligned
Thinker	#111827 (Surface)	#F9FAFB	Left-aligned

23.4 Settings — 8 Tabs

Tab	Key Settings
General	App language · Response language · Default Thinking Level · Default domain · Theme (Dark/Light) · Font size
Thinking & Pipeline	Auto-select Thinking Level · Show credit cost preview · Show pipeline stages · Signature Question on/off · Goal Dis
Memory	Project Memory on/off · Long-term Preference on/off · Decision Memory on/off · Clear all memory · Export all memo
Files & Uploads	Default action on code/document/image/ZIP upload · Auto-scan code for security · Warn before sending file to LLM

API & Webhook Specification

The Thinker AI API is available to Founder plan users only. Base URL: `https://api.thinker.ai/v1` · Sandbox: `https://sandbox.api.thinker.ai/v1` · Auth: Authorization: Bearer `tk_live_xxxx` · Rate limit: 60 req/min per API key.

24.1 Core Endpoints

Method	Endpoint	Purpose
POST	<code>/pipeline/run</code>	Run the full pipeline on a prompt. Returns <code>pipeline_id</code> immediately (async).
GET	<code>/pipeline/{id}</code>	Get status and result of a pipeline run.
POST	<code>/pipeline/{id}/respond</code>	Send user response to clarification/approval prompt mid-pipeline.
DELETE	<code>/pipeline/{id}</code>	Cancel a running pipeline.
GET	<code>/credits/balance</code>	Get current Think Credit balance and monthly usage.
GET	<code>/credits/history</code>	Paginated list of credit transactions.
GET	<code>/projects</code>	List all projects for this account.
GET	<code>/projects/{id}/versions</code>	List all saved versions of a project.
POST	<code>/files/upload</code>	Upload a file for use in a pipeline run.

24.2 Webhook Events

Event	When Fired
<code>pipeline.started</code>	Pipeline begins running
<code>pipeline.clarification_needed</code>	Clarification Agent needs user input
<code>pipeline.stage_complete</code>	Each pipeline stage completes
<code>pipeline.approval_needed</code>	Stage 4 approval required
<code>pipeline.complete</code>	Pipeline finishes successfully
<code>pipeline.failed</code>	Pipeline fails unrecoverably
<code>pipeline.halted</code>	Loop detection or pool exhaustion halts pipeline
<code>credits.low_balance</code>	Balance drops below threshold
<code>credits.depleted</code>	Balance reaches 0
<code>failover.occurred</code>	Model failover happened (informational)

Webhook security: every request includes a Thinker-Signature header (HMAC-SHA256). Thinker AI retries failed deliveries up to 5 times with exponential backoff (1m, 5m, 30m, 2h, 8h). After 5 failures, the endpoint is disabled and the founder is notified.

Testing & QA Strategy

25.1 Testing Layers

Layer	What Is Tested	When
Unit tests	Thinker Core routing logic, credit calculations, loop detection — not LLM calls	Every commit (CI)
Agent mock tests	Each agent's input→output contract using deterministic mocks	Every commit (CI)
Integration tests	10 fixed test prompts against real models, scored by Judge Agent. Pass rate > 70%	Every staging deploy
Failover tests	Primary model blocked — pipeline must complete via secondary with no delay	Weekly automated
Credit accuracy tests	Each action type credited exactly per Section 21.3 table	Every staging deploy
Security tests	Upload test files: malware pattern, executable, oversized, credential-containing	Every staging deploy
Manual QA scenarios	End-to-end user flows that cannot be automated (see Section 25.2)	Before each release

25.2 Key Manual QA Scenarios

Scenario	Key Assertions
A — Simple chat	■ Low Thinking auto-selected · 1 credit deducted · No clarification asked · Answer in <5s
B — Project build	Signature Question asked · Goal Discovery runs · All 5 stages visible · ZIP downloadable · Credits match estimate
C — Vague request	Intent confidence below 70% → Goal Discovery Mode · Questions asked one at a time · No Planner until discovery
D — Failover invisible	Primary model blocked · Build completes · No error shown to user · failover_log has one entry
E — Credit gate	Balance 3 credits · High Thinking request (10 credits) → confirmation modal → 'Insufficient' if confirmed
F — Code review upload	File received card shown · Auto-review (5 credits) · Bug identified · Fix offered · Fixed file downloadable
G — RTL language	Write in Arabic · All responses in Arabic · UI right-aligned · Code stays in English · README in Arabic
H — Loop detection	Force Reviewer to always return retry · Builder retried exactly 3 times · Halt on 4th · State saved

Error Codes & Messages

Every error has a machine-readable code, a user-facing message (non-technical, in the user's language), and a developer-facing message (English, technical). No provider or model name ever appears in any error message.

Code	User Message	Developer Message
AUTH_001	Please sign in to continue.	Missing or invalid Authorization header.
AUTH_002	Your session has expired. Please sign in again.	JWT token expired.
AUTH_004	This API key is invalid or has been revoked.	API key not found or revoked in database.
CREDIT_001	You don't have enough Think Credits. You have {balance} credits and need {required}.	Credit balance is insufficient.
CREDIT_002	Your Think Credits have run out. Top up to continue. Credit balance is 0.	Credit balance is 0.
PLAN_001	Strategy Agent is available on Pro and above.	Feature strategy_agent not available on plan: free_trial.
PLAN_002	Consensus Thinking is available on the Founder plan	Feature consensus_thinking not available on plan: pro.
PLAN_003	You've reached your monthly project limit. Upgrade to	Monthly_project_count >= plan_limit.
PIPELINE_001	I need a bit more information before I can start.	Pipeline rejected: requirements_complete=false.
PIPELINE_002	Something went wrong. Your work up to this point is saved.	Unhandled exception in pipeline at stage={stage_name}.
PIPELINE_003	I ran into a difficulty on this step. Your work is saved.	All models in pool exhausted for agent={agent_name}.
PIPELINE_004	This step has been retried too many times. Let's try describing the problem differently.	Step_limit_exceeded_in_pipeline.
AGENT_001	Still working on this — taking a moment longer than usual.	Model timeout after 5s. Failover initiated.
FILE_001	That file type isn't supported.	Unsupported MIME type: {mime_type}.
FILE_002	That file is too large. Max size is {limit}MB.	File size {size} exceeds limit for type {category}.
FILE_003	This file type can't be uploaded for security reasons.	Blocked file extension: {ext}.
FILE_004	This code file appears to contain sensitive data. Please remove it before uploading.	Credential pattern detected in code file.
FILE_005	This code contains patterns that look potentially harmful.	Malicious code pattern detected: {category}.
API_001	API rate limit reached. Please wait before making more requests.	Rate limits exceeded: {count} requests in 60s.
WEBHOOK_001	N/A	Webhook delivery failed after 5 attempts. Endpoint disabled, founder notified.

Analytics & Business Metrics

27.1 Key Analytics Events

Event	Fired When	Key Properties
session_started	User opens the app	plan, language, domain_selected, is_returning_user
message_sent	User sends any message	thinking_level, has_file_attached, domain
pipeline_started	Full pipeline begins	thinking_level, domain, intent_type, credits_estimated
clarification_shown	Questions returned to user	question_count, clarification_depth, discovery_layers
pipeline_complete	Pipeline finishes successfully	thinking_level, credits_used, duration_seconds, failover_count
pipeline_failed	Pipeline fails	failed_at_stage, error_code, credits_charged
file_uploaded	File upload completes	file_type_category, size_bytes, action_intent
upgrade_prompt_shown	Upgrade CTA shown	trigger_error_code, feature_gated, current_plan
upgrade_clicked	User clicks upgrade CTA	from_plan, to_plan, trigger_feature
credits_purchased	User buys extra credits	credits_amount, price_paid, trigger
failover_occurred	Model failover in pipeline	agent_name, attempt_number, peer_audit_needed

27.2 Business Health Metrics

Metric	V1 Target	Warning Threshold
Pipeline completion rate	≥ 70%	< 50% = reliability problem
Free → Pro conversion (30 days)	≥ 8%	< 3% = value not clear
Pro → Founder upgrade (60 days)	≥ 5%	< 2% = Founder plan unclear
Monthly churn (Pro)	≤ 5%	> 10% = product not delivering
Avg credits / High Thinking build	40–80 credits	> 120 = cost unsustainable
Operator LLM cost / High build	≤ \$0.60	> \$1.20 = margin unsustainable
Failover rate	≤ 5%	> 15% = provider reliability issue
Clarification abandonment	≤ 20%	> 35% = too many questions
Upgrade prompt CTR	≥ 12%	< 5% = prompts not persuasive
Support ticket rate	≤ 2 per 100 users/wk	> 8 = UX or reliability problem

27.3 V1 Success Definition

V1 is considered successful if, 60 days after launch, ALL of the following are true: pipeline completion $\geq 70\%$, Free→Pro conversion $\geq 8\%$, Pro churn $\leq 5\%$, operator cost per High Thinking build $\leq \$0.60$, failover rate $\leq 5\%$, and positive qualitative user feedback is visible.

Brand Identity Guidelines

28.1 Brand Personality

Trait	What It Means in Practice
Intelligent	Reasons, validates, and challenges assumptions — never guesses
Trustworthy	Every output reviewed by multiple agents before the user sees it
Minimal	No noise, no clutter — every element serves a purpose
Premium	The experience feels crafted, not generated
Futuristic	Ahead of the curve in product, design, and thinking
Calm	Never urgent, never anxious — steady, focused intelligence

28.2 Taglines

Rating	Tagline	Best For
★★★★★ Primary	Think Beyond Intelligence	App, splash screen, primary marketing
★★★★ Secondary	Many Minds. One Intelligence.	Architecture, investor pitch
★★★ Secondary	The AI That Thinks Before It Builds.	Product description, onboarding

28.3 Color System

Name	Hex	Use	Do Not Use For
Deep Teal	#0B6E69	Primary brand color — buttons, links, active states	Body text, large backgrounds
Teal Light	#14B8A6	Core glow, gradient end, accent highlights, code blocks	Backgrounds, paragraph text
Dark BG	#0B1220	App background, splash screen, full-bleed sections	Cards, modals
Surface	#111827	Cards, sidebars, modals, input fields	Main background
Divider	#1F2937	Dividers, card outlines, input borders	Text, icons
Text White	#F9FAFB	Headings, labels, important body text	Disabled states
Text Mid	#94A3B8	Body text, descriptions, secondary content	Headings
Gradient	#0B6E69→#14B8A6	Primary CTA buttons, progress bars	Backgrounds, body text

28.4 Typography

Font	Role	Sizes	Weight
Space Grotesk	All headings, navigation, brand display	Display 48 · H1 32 · H2 24 · H3 20 · H4 16px	Bold 700 / SemiBold 600

Font	Role	Sizes	Weight
Inter	All body text, descriptions, captions	Only 16 · Body 14 · Caption 12 · Label 11px	Regular 400 / Medium 500
JetBrains Mono	All code blocks, API output, keys	13px	Regular 400

28.5 Logo Meaning

Element	Meaning
Center Core	Thinker Core — the orchestration brain of the system
Outer Nodes (×6)	The 6 primary AI Agent roles working in parallel
Connection Lines	Agent collaboration and data flow between all agents
Core Color (#0B6E69)	Active, focused intelligence — the system is thinking
Whole Symbol	Unified Intelligence — many minds, one trusted outcome

28.6 Logo Rules

Always:

- ✓ Use on a dark background (#0B1220 preferred)
- ✓ Use the SVG master file for all digital surfaces
- ✓ Maintain equal clear space (minimum: logo width × 0.3)
- ✓ Keep perfect symmetry — all 6 nodes equally spaced

Never:

- ✗ Add drop shadows, 3D effects, or emboss
- ✗ Use neon or rainbow colors — one teal color only
- ✗ Apply glow to outer nodes — Center Core only
- ✗ Place on a light or white background
- ✗ Rearrange, add, or remove nodes
- ✗ Put text inside the logo symbol

28.7 UI Design System

Property	Value
Default theme	Dark Mode (Light mode available but secondary)
Button radius	16px
Card radius	20px · 1px border · No shadows
Shadow	None — flat design system
Transition	200–250ms ease-out
Icons	Lucide Icons — outline only — 1.5px stroke — 24×24px base

Property	Value
Loading	3 dots: ● → ●● → ●●● at 400ms intervals — never a spinner

28.8 Brand Voice

■ Never Say	■ Thinker AI Says
Here is your answer.	I analyzed your goal. Here's what I found.
Great question! I'd be happy to help!	Let me think through this.
Processing your request...	Analyzing your goal...
Done! Your project is ready.	Build complete. Reviewed by 4 agents. Here's what was built and why.

28.9 App Open Animation — 2–3 Seconds

Time	What Happens
0ms	Black screen.
0–300ms	6 outer nodes fade in, 50ms apart, clockwise
300–600ms	Connection lines draw from nodes toward center
600–1000ms	Center Core brightens — Deep Teal glow pulses outward once
1000–1500ms	'Thinker AI' fades in letter by letter below logo
1500–2000ms	Tagline fades in: 'Think Beyond Intelligence'
2000–2500ms	Brief hold. Fade to Home Screen.

28.10 Do's & Don'ts — Quick Reference

■ Always Do	■ Never Do
Use #0B1220 as the app background	Never use logo on white or light background
Use Space Grotesk Bold for all headings	Never add drop shadows to any element
Use Inter Regular for all body text	Never use 3D or neon effects
Use Lucide outline icons at 1.5px stroke	Never use filled icons — outline only
Show thinking progress — all pipeline stages	Never put text inside the logo symbol
Use the gradient only on primary CTA buttons	Never say 'Great question!' or 'Happy to help'
Use 3 dots (●●●) for loading — never a spinner	Never show provider names (Claude, GPT, etc.)
Keep Core glow on Center Core only	Never use the gradient as a page background
Animate at 200–250ms — slow, smooth, no bounce	Never use a spinner for loading
Keep sound off by default	Never bounce or spring any animation

Think Beyond Intelligence

Thinker AI Complete Specification — Sections 1–28 · June 2026